

Aprendiendo a jugar a Oteló: Agente Inteligente con Búsqueda Adversaria y Deep Learning

1st Javier Luque Ruiz

Grado en Ingeniería Informática - Ingeniería del Software

Universidad de Sevilla

Sevilla, España

javluqrui@alum.us.es

Resumen—En este trabajo se explorará el desarrollo de un agente inteligente para el clásico juego de tablero Oteló (Reversi). El objetivo principal es construir un jugador capaz de tomar decisiones estratégicas mediante la implementación del algoritmo clásico de Minimax con poda alfa-beta para explorar el espacio de decisiones. Adicionalmente, se integrará una red neuronal profunda entrenada mediante autojuego para proporcionar una heurística de evaluación de las posiciones intermedias, reemplazando las funciones de evaluación manuales clásicas.

Index Terms—Inteligencia Artificial, Oteló, Reversi, Minimax, Poda Alfa-Beta, Deep Learning, Python.

I. INTRODUCCIÓN

El desarrollo de agentes inteligentes capaces de tomar decisiones estratégicas en juegos de tablero ha sido uno de los grandes hitos de la Inteligencia Artificial. Inspirado en los éxitos recientes de sistemas basados en aprendizaje profundo, este trabajo explora la combinación de algoritmos de búsqueda clásica con técnicas modernas de redes neuronales.

El objetivo principal de este proyecto es construir un agente inteligente para el clásico juego de Oteló (también conocido como Reversi). Específicamente, se aborda la variante del proyecto correspondiente a la convocatoria de julio, la cual requiere la implementación del algoritmo Minimax con poda alfa-beta para explorar el espacio de decisiones. La aportación central del sistema consiste en sustituir las funciones heurísticas manuales por una red neuronal profunda, la cual aprenderá a evaluar las posiciones intermedias del tablero basándose en un conjunto de datos generado mediante partidas de autojuego.

II. PREELIMINARES

II-A. El juego de Oteló

Oteló es un juego de estrategia de suma cero con información perfecta para dos jugadores, el cual se disputa sobre un tablero de 8×8 casillas, siguiendo el reglamento oficial de la Federación Mundial de Oteló [?]. La partida inicializa con una disposición central de cuatro fichas cruzadas (dos blancas y dos negras), cediendo el primer movimiento al jugador con las piezas negras.

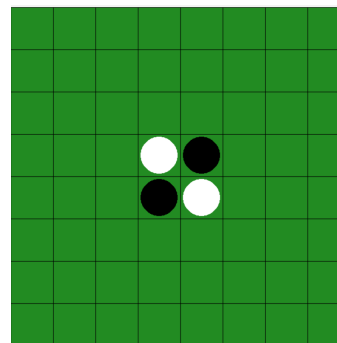


Figura 1. Tablero inicial de Oteló con la disposición central de fichas.

La mecánica central del juego es el *flanqueo*: un movimiento legal consiste en colocar una ficha en una casilla vacía de forma que, en línea recta (horizontal, vertical o diagonal), una o más piezas del oponente queden atrapadas entre la nueva ficha y otra pieza del jugador activo. Las fichas flanqueadas son capturadas y cambian de color.

Para ilustrar esta mecánica, la Fig. ?? muestra las casillas vacías donde el jugador con fichas negras puede colocar su pieza en el primer turno de la partida. Cada una de estas posiciones permite flanquear al menos una ficha blanca, cumpliendo así con la regla de legalidad del movimiento. Por el contrario, las casillas marcadas en rojo representan un movimiento ilegal, ya que, aunque estén adyacentes a fichas del oponente, colocar una ficha ahí no lograría atrapar ninguna ficha blanca entre dos fichas negras.

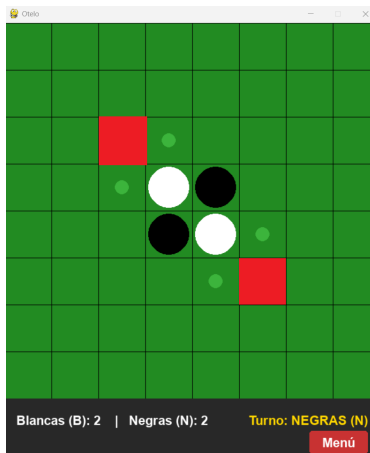


Figura 2. Ejemplo de movimientos: flaqueo válido (verde) y movimiento ilegal (rojo).

Siguiendo con el escenario planteado, supongamos que el jugador de piezas negras decide ejecutar el movimiento válido situado más a la izquierda del tablero. Al colocar su ficha en esa posición, logra atrapar una pieza blanca del oponente contra otra de sus propias fichas negras situadas en el centro.

Como dicta la regla del flaqueo, esta ficha blanca es capturada y revertida al color negro. El resultado de esta acción se ilustra en la Fig. ?? . Tras este cambio en el tablero, el turno pasa inmediatamente al jugador de piezas blancas, continuando así el flujo natural de la partida.

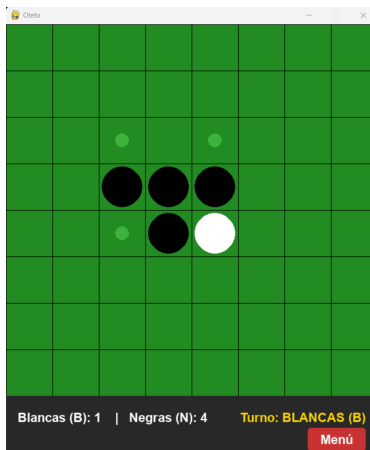


Figura 3. Estado del tablero tras ejecutar el flaqueo. La ficha blanca capturada cambia a color negro.

Esta restricción de movimientos obliga a los jugadores a desarrollar estrategias de posicionamiento a largo plazo. En este sentido, el control de las esquinas del tablero resulta vital, ya que una ficha colocada en una de las cuatro esquinas jamás podrá ser flanqueada por el oponente, garantizando su permanencia hasta el final de la partida. Del mismo modo, limitar la movilidad del rival (reduciendo sus opciones legales) es una táctica fundamental para forzarle a realizar movimientos perjudiciales.

Si en algún momento un jugador carece de movimientos legales que cumplan esta regla de flaqueo, está obligado a pasar su turno. La partida concluye cuando el tablero se llena o ningún jugador puede realizar un movimiento válido, resultando ganador aquel con mayor número de fichas de su color.

II-B. Búsqueda Adversaria y Poda Alfa-Beta

El desarrollo de un agente capaz de competir en juegos de tablero bidireccionales y de suma cero se fundamenta tradicionalmente en la teoría de la búsqueda adversaria [?]. En este contexto, el algoritmo Minimax constituye la técnica esencial para modelar la toma de decisiones estratégicas. Este enfoque asume que ambos jugadores actúan de forma completamente óptima bajo roles contrapuestos: el agente inteligente adopta el rol de maximizador (MAX), cuyo objetivo es seleccionar el movimiento que maximice su recompensa final, mientras que el oponente actúa como minimizador (MIN), intentando reducir a toda costa el beneficio de MAX.

El proceso opera de forma recursiva construyendo un árbol de juego, donde los nodos representan los estados del tablero y las aristas corresponden a los movimientos legales posibles. Los valores de utilidad se calculan desde los nodos terminales (hojas) y se propagan hacia arriba a través de los niveles del árbol: un nodo en un nivel MAX heredará el valor máximo de sus hijos, mientras que un nodo en un nivel MIN heredará el valor mínimo.

Sin embargo, el espacio de estados de Othello es masivo, con un factor de ramificación medio elevado. Explorar el árbol completo hasta el final de la partida desde los primeros turnos es computacionalmente inabarcable en tiempo real. Por ello, se implementa una estrategia de búsqueda con corte de profundidad (*cutting-off search*). Al alcanzar un límite preestablecido de niveles (profundidad d), la simulación se detiene de manera controlada y se invoca una función heurística de evaluación numérica. Esta función estima de forma estática la calidad de la posición actual del tablero para MAX, sustituyendo el valor de utilidad real del fin de la partida.

Para mitigar el coste computacional y permitir exploraciones más profundas en menores tiempos de respuesta, se integra la optimización de la poda alfa-beta. Esta técnica mantiene el control de los peores escenarios asumibles por cada jugador durante la exploración recursiva mediante dos variables adaptativas:

- α : Representa la mejor opción (el valor más alto) que el jugador MAX ha logrado asegurar hasta el momento a lo largo del camino de búsqueda.
- β : Representa la mejor opción (el valor más bajo) que el jugador MIN ha conseguido asegurar hasta el momento.

A medida que el algoritmo recorre el árbol en profundidad, actualiza estos valores. La poda (o descarte matemático de ramas) se ejecuta inmediatamente en el momento en que se cumple la condición:

$$\alpha \geq \beta \quad (1)$$

Esta desigualdad implica que el jugador actual puede forzar un resultado mejor en otra rama previamente explorada, por lo que el oponente ideal jamás permitiría alcanzar el estado actual. Descartar estas ramas irrelevantes reduce drásticamente el número de nodos evaluados sin alterar en absoluto el movimiento óptimo final que devolvería el algoritmo Minimax puro.

Para comprender visualmente este proceso de descarte, la Fig ?? ilustra el escenario conceptual diseñado para el árbol de decisión. Como se puede observar, tras evaluar el primer hijo de un nodo MIN y obtener un valor que resulta inferior o igual al valor α asegurado por MAX, se activa inmediatamente el criterio de poda, omitiendo la exploración de los nodos restantes (marcados con un descarte explícito).

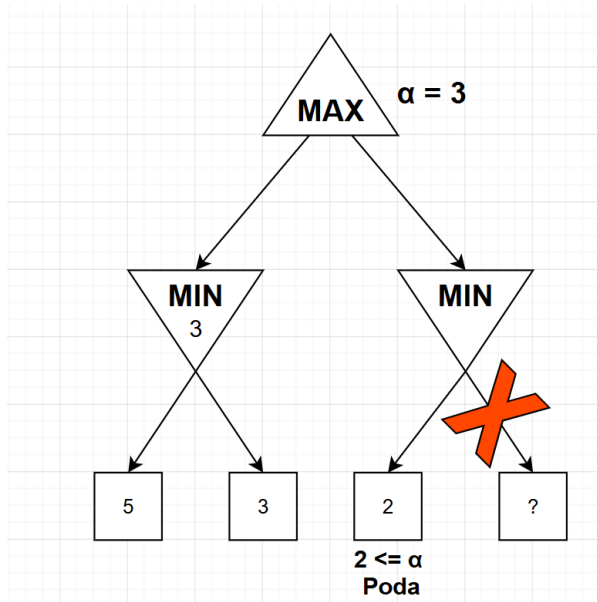


Figura 4. Ejemplo conceptual de poda alfa-beta en un árbol de búsqueda. Los nodos descartados se marcan con una cruz roja, indicando que no se evaluarán debido a la condición $\alpha \geq \beta$.

II-C. Redes Neuronales y Deep Learning

Las redes neuronales artificiales son modelos computacionales inspirados en la estructura y el funcionamiento del cerebro humano, diseñados para aproximar funciones matemáticas complejas a partir de datos empíricos [?]. En particular, las redes con alimentación hacia adelante (*feedforward neural networks*) se estructuran mediante unidades básicas denominadas neuronas, las cuales se organizan en capas secuenciales (entrada, ocultas y salida) [?].

Cada conexión entre neuronas posee un peso numérico asociado que modula la señal transmitida. Durante la fase de aprendizaje, estos pesos se ajustan iterativamente mediante algoritmos de optimización basados en el cálculo del gradiente, buscando minimizar una función de error o pérdida que mide la discrepancia entre la salida real de la red y el valor esperado [?].

En el contexto de este proyecto, el Aprendizaje Profundo (*Deep Learning*) se introduce como una evolución del agente clásico de Minimax. En lugar de depender de una función heurística manual estática (como el simple recuento de fichas), se entrena una red neuronal profunda para evaluar la calidad de los estados intermedios del tablero y ofrecer una estimación más flexible de la posición.

La red diseñada toma como entrada la representación matricial del tablero de 8×8 y, tras procesar la información a través de sus capas ocultas mediante funciones de activación no lineales, devuelve un único valor continuo en el intervalo $[-1, 1]$. Esta salida representa la estimación de la posición: un valor cercano a 1 indica una alta probabilidad de victoria para el jugador activo, mientras que un valor cercano a -1 augura una derrota inminente.

III. IMPLEMENTACIÓN

III-A. Motor del Juego: La clase Oteló

El núcleo lógico del proyecto se ha encapsulado en la clase *Oteló*, desarrollada íntegramente en Python. El diseño de este motor se fundamenta en un principio de responsabilidad única, separando completamente las reglas del juego de la interfaz gráfica y de la inteligencia artificial. Esta arquitectura modular resulta crucial para la posterior fase de autójuego, ya que permite simular miles de partidas en segundo plano sin el coste computacional de renderizar gráficos.

III-A1. Representación del Estado: Para la representación del tablero de Oteló se ha seleccionado una matriz bidimensional de 8×8 elementos utilizando la librería NumPy. Esta decisión de diseño frente a una representación basada en listas responde a la necesidad de gestionar los estados de forma eficiente y preparar los datos para su posterior tratamiento en la fase de Deep Learning.

De acuerdo con las especificaciones del enunciado del proyecto, se ha optado por utilizar la siguiente convención numérica para representar los distintos estados de las casillas:

- 0: Casilla vacía.
- 1: Casilla ocupada por una ficha blanca.
- 2: Casilla ocupada por una ficha negra.

El estado inicial se configura en el constructor de la clase, estableciendo la disposición central cruzada clásica y cediendo el primer turno reglamentario a las piezas negras, tal y como se muestra en el Listado ??.

```

1 def __init__(self):
2     # Inicialización del tablero de Othello como una
3     # matriz de 8x8 con ceros
4     self.tablero = np.zeros((8, 8), dtype=int)
5
6     # Posición inicial de las fichas en el tablero
7     self.tablero[3][3] = 1
8     self.tablero[3][4] = 2
9     self.tablero[4][3] = 2
10    self.tablero[4][4] = 1
11
12    # Inician las fichas negras
13    self.jugador_actual = 2

```

Listing 1. Inicialización del estado del tablero usando NumPy.

III-A2. Lógica de Juego y Validación de Movimientos:

El mayor desafío algorítmico del motor radica en la validación de los movimientos (método `es_movimiento_valido(fila, columna, jugador)`). Dado que la mecánica de flanqueo exige evaluar casillas en línea recta, se ha implementado un escaneo vectorial utilizando un conjunto estático de ocho tuplas que representan los desplazamientos espaciales en el tablero (ejes horizontal, vertical y las cuatro diagonales):

$$D = \{(0, 1), (1, 1), (1, 0), (1, -1), (0, -1), (-1, -1), (-1, 0), (-1, 1)\}$$

Cuando un jugador solicita colocar una ficha en las coordenadas (f, c) , el algoritmo primero verifica que la casilla objetivo contenga un 0, es decir, que esté vacía. Posteriormente, itera sobre cada vector de dirección $\vec{v} \in D$, avanzando paso a paso. Un movimiento se marca como válido si, y solo si, en al menos una dirección se detecta una secuencia ininterrumpida de una o más fichas rivales seguida inmediatamente por una ficha del jugador activo, sin rebasar los límites físicos del tablero definidos por la matriz de 8×8 .

III-A3. Transiciones de Estado y Condiciones de Victoria: Una vez que la validación confirma la legalidad, el método `ejecutar_movimiento(fila, columna, jugador)` aplica la transición del estado. Para ello, reaprovecha la lógica del escaneo direccional para recopilar las coordenadas de todas las piezas atrapadas y procede a mutar sus valores en la matriz, invirtiendo su color. Finalmente, actualiza el puntero de `jugador_actual`, pasando el turno al siguiente jugador.

Para garantizar el correcto desarrollo de la partida y el funcionamiento futuro del algoritmo Minimax, el motor implementa mecanismos precisos de detección de estados terminales. El método `obtener_movimientos_validos(jugador)` explora sistemáticamente las 64 casillas devolviendo una lista de tuplas con los flanqueos legales. Si al evaluar esta función para ambos jugadores el resultado es una lista vacía, el método `es_fin_de_juego` dictamina el final de la partida, procediendo al recuento numérico de los valores en la matriz para declarar al ganador final.

III-B. Interfaz Gráfica y Máquina de Estados

La interacción con el usuario y la visualización del estado del juego se gestionan en el archivo `main.py` mediante la API gráfica de PyGame. Para evitar el acoplamiento de código y controlar de forma robusta el flujo de la aplicación, se ha diseñado una Arquitectura basada en una Máquina de Estados Estricta. El bucle principal del juego (*Game Loop*) evalúa de forma continua los eventos del sistema y renderiza la interfaz basándose en el estado activo:

- **MENU:** Pantalla de inicio que presenta el título y el botón para iniciar una nueva partida.
- **JUGANDO:** Estado activo durante el juego, donde se procesan los movimientos de los jugadores y se actualiza la visualización del tablero. Cuenta con un panel inferior que muestra información relevante, como el turno actual y el marcador de fichas.
- **PASAR TURNO:** Estado crítico de validación que se activa cuando un jugador no tiene movimientos válidos disponibles, pero la partida continúa. En este estado, se notifica al jugador que está obligado a pasar turno mediante una confirmación explícita.
- **FIN:** Estado final que congela el estado del tablero y muestra el resultado de la partida, indicando el ganador o si ha habido un empate, así como el recuento final de fichas.

III-C. Agente Inteligente Clásico: Minimax con Poda Alfa-Beta

La capacidad de decisión lógica del sistema se ha encapsulado en la clase `AgenteMinimax`. Esta entidad implementa el algoritmo de búsqueda adversaria descrito en la Sección II-B, actuando como el jugador maximizador (MAX) que busca asegurar la victoria frente al jugador humano u otro agente (MIN).

La exploración del árbol de estados se gestiona mediante el método recursivo `_minimax_alfa_beta(partida, profundida, alfa, beta, maximizando)`. Para evitar la corrupción del estado de la partida real durante la simulación de futuros turnos, el agente emplea clonación profunda de objetos (`copy.deepcopy`) en cada ramificación. Cuando la recursividad alcanza el límite definido por `profundidad_maxima`, la simulación se detiene y se evalúa el estado del tablero mediante una función heurística manual. En esta versión clásica, la heurística consiste en calcular la diferencia neta de fichas entre el agente y el oponente, favoreciendo aquellos estados que maximicen el dominio territorial de la IA.

Durante la implementación algorítmica, se identificaron y resolvieron dos desafíos arquitectónicos críticos para asegurar la viabilidad del agente:

- **Gestión de turnos nulos en simulación:** En partidas avanzadas de Othello, es habitual alcanzar estados donde

un jugador carece de movimientos válidos. Para solucionar este problema, se implementó una lógica condicional en el flujo recursivo (ver Listado ??). Si el método `obtener_movimientos_validos` devuelve una lista vacía, el agente no aborta la ejecución; en su lugar, cede el turno de forma virtual invocando de nuevo a la función con el rol del oponente invertido. Para certificar la estabilidad de esta solución frente a dichos escenarios límite, se desarrolló una suite de pruebas unitarias (`test_minimax.py`) basada en `unittest`, garantizando la ausencia de bucles infinitos.

- **Ruptura del determinismo estricto:** Al depender de una heurística discreta basada en el mero recuento de fichas, el agente clásico tendía a generar patrones de juego repetitivos ante situaciones de empate heurístico. Para solucionarlo, el método `obtener_mejor_movimiento` recopila todos los movimientos que alcanzan la puntuación máxima (`max_eval`) y selecciona uno aleatoriamente mediante `random.choice`. Esta introducción de pseudoaleatoriedad es un requisito técnico fundamental para garantizar una exploración heterogénea del espacio de estados, preparando al motor para la futura fase de generación de datos por autojuego.

```

1 # --- DENTRO DEL METODO _MINIMAX_ALFA_BETA ---
2 if es_maximizando:
3     movimientos = partida.
4     obtener_movimientos_validos(self.jugadorIA)
5
6     # Prevencion de bloqueos: si MAX no puede mover,
7     # cede el turno a MIN
8     if len(movimientos) == 0:
9         return self._minimax_alfa_beta(partida,
10 profundidad-1, alfa, beta, False)
11
12 max_eval = -float('inf')
13 for movimiento in movimientos:
14     partida_copia = copy.deepcopy(partida)
15     f, c = movimiento
16     partida_copia.ejecutar_movimiento(f, c, self
17 .jugadorIA)
18
19     evaluacion = self._minimax_alfa_beta(
20 partida_copia, profundidad - 1, alfa, beta,
21 False)
22
23     max_eval = max(max_eval, evaluacion)
24     alfa = max(alfa, max_eval)
25     if beta <= alfa:
26         break # Poda Alfa-Beta
27 return max_eval

```

Listing 2. Mecanismo recursivo de Minimax y control de turnos nulos (virtual pass).

Cabe destacar que la clase `AgenteMinimax` ha sido diseñada siguiendo un patrón de estrategia. Mediante el parámetro de inicialización `usar_red`, el motor es capaz de alterar dinámicamente su comportamiento, sustituyendo el conteo manual de fichas por inferencias procedentes de un modelo de Deep Learning, cuya concepción se detallará en las secciones posteriores.

III-D. Generación del Conjunto de Datos mediante Autojuego

Para entrenar la red neuronal, es necesario disponer de un conjunto de datos etiquetado que relacione estados del tablero con la calidad de la posición para el jugador activo. Dado que no existen bases de datos públicas de partidas de Oteho, se ha desarrollado un módulo de autojuego (`generar_datos.py`) que enfrenta al agente Minimax contra sí mismo.

Se diseñó un entorno de simulación aislado de la interfaz gráfica donde dos agentes Minimax (configurados a una profundidad de búsqueda de 3 niveles) compiten entre sí. Durante la ejecución de cada partida, se registran los estados del tablero antes de cada movimiento junto al jugador activo.

Una vez finalizada la partida, se determina el ganador y se recorre el historial de estados para etiquetar cada posición con el resultado final desde la perspectiva del jugador activo en ese momento:

- **+1:** Si el jugador activo en ese estado ganó la partida.
- **0:** Si la partida terminó en empate.
- **-1:** Si el jugador activo en ese estado perdió la partida.

El Listado ?? ilustra la implementación algorítmica de este proceso de retropropagación de la recompensa.

```

1 # Evaluacion del ganador (1: Blancas, 2: Negras, 0:
2 # Empate)
3 blancas, negras = partida.calcular_puntuacion()
4 ganador = 1 if blancas > negras else (2 if negras >
5 blancas else 0)
6
7 # Etiquetado del historial temporal
8 for estado_tablero, jugador in historia_partida:
9     if ganador == 0:
10         etiqueta = 0
11     elif jugador == ganador:
12         etiqueta = 1
13     else:
14         etiqueta = -1
15
16 tableros_totales.append(estado_tablero)
17 etiquetas_totales.append(etiqueta)

```

Listing 3. Lógica de etiquetado de estados intermedios basándose en el resultado final.

Aprovechando la aleatoriedad introducida previamente en el agente para romper el determinismo, se simulaban 1000 partidas completas de forma automatizada, generando una base de datos heterogénea de aproximadamente 60.000 tableros.

Para optimizar el rendimiento de lectura durante la fase de entrenamiento de la red neuronal, las estructuras de datos se transformaron en tensores y se exportaron en formato binario nativo (`.npy`) haciendo uso de la librería `NumPy`, separando las características de entrada (matrices de estado de los tableros) de las etiquetas de salida.

III-E. Arquitectura Base y Entrenamiento de la Red Neuronal

Una vez generado el conjunto de datos mediante autojuego, se procedió a diseñar y entrenar una red neuronal profunda para aprender a evaluar las posiciones del tablero. Para ello, se utilizó la interfaz de Keras sobre el motor TensorFlow.

Como paso previo al diseño de la red, se aplicó una técnica de retención de datos. Utilizando la librería `scikit-learn`, el conjunto de datos global se dividió en un 80 % para el entrenamiento del modelo y un 20 % para validación. Para garantizar la reproducibilidad de los experimentos posteriores, se fijó una semilla de pseudoaleatoriedad (`random_state = 42`).

Para la primera iteración de la inteligencia artificial, denominada como la arquitectura base o **Modelo A**, se implementó un Perceptrón Multicapa (MLP) mediante la clase `Sequential` de Keras. El diseño arquitectónico se estructuró de la siguiente manera:

- **Capa de Entrada y Aplanado:** Recibe la matriz bidimensional del tablero de 8×8 casillas, la cual es procesada mediante una capa de aplanado (`Flatten`) para convertirla en un vector unidimensional de 64 elementos.
- **Capas Ocultas:** Dos capas densas (`Dense`) con 64 y 32 neuronas respectivamente, ambas con función de activación `ReLU` para introducir no linealidad y evitar la saturación de gradientes.
- **Capa de Salida:** Una única neurona con función de activación de tangente hiperbólica (`tanh`). Esta función es matemáticamente idónea para este dominio, ya que acota las predicciones de la red en un rango continuo de $[-1, 1]$, alineándose perfectamente con el etiquetado diseñado en la fase de autojuego.

El Listado ?? muestra la implementación de esta topología base.

```
1 def entrenar_modelo(nombre_archivo,
2   activacion_oculta, optimizador):
3     print(f"Entrenando {nombre_archivo}...")
4
5     red_otelo = Sequential()
6     red_otelo.add(Input(shape=(8, 8)))
7     red_otelo.add(Flatten())
8     red_otelo.add(Dense(64, activation=
9       activacion_oculta))
10    red_otelo.add(Dense(32, activation=
11      activacion_oculta))
12    red_otelo.add(Dense(1, activation='tanh'))
13
14    red_otelo.compile(
15      optimizer=optimizador,
16      loss='mean_squared_error',
17      metrics=['mean_absolute_error']
18    )
19
20    red_otelo.fit(
```

```
18     atributos_entrenamiento,
19     objetivo_entrenamiento,
20     epochs=50,
21     batch_size=256,
22     validation_data=(atributos_prueba,
23       objetivo_prueba),
24     verbose=1
25   )
26
27   red_otelo.save(f"modelos/{nombre_archivo}.keras"
28 )
29   print(f"Guardado {nombre_archivo}.keras")
30
31   entrenar_modelo("otelo_A", "relu", SGD(learning_rate
32     =0.01))
```

Listing 4. Construcción y compilación de la arquitectura base de la red neuronal mediante Keras.

Para el proceso de optimización, se seleccionó el algoritmo de Descenso Estocástico por el Gradiente (SGD) con un factor de aprendizaje fijo de 0.01. La función de pérdida elegida fue el Error Cuadrático Medio (MSE), evaluada conjuntamente con la métrica del Error Absoluto Medio (MAE). El entrenamiento se llevó a cabo durante 50 épocas con un tamaño de lote de 256 ejemplos, aplicando una división de 80 % para entrenamiento y 20 % para validación.

IV. PRUEBAS Y EXPERIMENTACIÓN

IV-A. Estudio de Hiperparámetros y Selección del Modelo

Con el objetivo de optimizar la capacidad de generalización del modelo, en la fase de pruebas no se entrenó un único modelo, sino que se realizó un estudio comparativo evaluando tres configuraciones distintas de hiperparámetros.

Para garantizar la reproducibilidad del experimento, se fijaron las semillas de pseudoaleatoriedad (`seed = 42`) en las librerías `NumPy` y `TensorFlow`, asegurando que el reparto del 20 % de los datos de prueba fuera idéntico para todos los candidatos. Los tres modelos evaluados, todos con una estructura de 64 y 32 neuronas en sus capas ocultas, fueron los siguientes:

- **Modelo A:** Arquitectura base con función de activación *ReLU* y optimizador *SGD* con tasa de aprendizaje de 0.01.
- **Modelo B:** Mismo optimizador, pero sustituyendo *ReLU* por la función de activación *Sigmoide* en las capas ocultas.
- **Modelo c:** Arquitectura base (*ReLU*), pero sustituyendo el optimizador clásico por el algoritmo *Adam* con tasa de aprendizaje de 0.001, buscando una convergencia más rápida y estable.

Tras someter a los tres modelos a 50 épocas de entrenamiento, los resultados del error absoluto medio (MAE) sobre el conjunto de validación fueron los siguientes:

- Modelo A (Base): MAE de 0.9848
- Modelo B (Sigmoide): MAE de 0.9797
- Modelo C (Adam): MAE de 1.0384

El **Modelo C** experimentó un fenómeno de sobreajuste, con un error de validación superior al de entrenamiento, lo que indica que el optimizador *Adam* no fue capaz de generalizar adecuadamente en este caso. Por otro lado, el

Modelo B mostró una ligera mejora respecto al modelo base. No obstante, el error cercano a 1.0 sugiere que la red no está aprendiendo patrones significativos y, ante la duda, tiende a predecir la situación del tablero como un empate (0). Esta limitación está relacionada con el uso de la capa *Flatten*, que no preserva la estructura espacial del tablero, dificultando que la red capture relaciones locales entre las fichas.

Como prueba final para descartar que el error del modelo se debiera a una falta de capacidad paramétrica o a un entrenamiento demasiado corto, se diseñó un experimento límite (Modelo D). Se construyó una topología masiva, incluyendo capas de *BatchNormalization* y *Dropout*, y se extendió el entrenamiento hasta las 500 épocas. Los resultados confirmaron el diagnóstico previo: la red alcanzó un error de entrenamiento bastante mejor que los previos (MAE de 0.7739), pero falló drásticamente en el conjunto de validación, disparando su error hasta 1.0058. Este comportamiento evidencia otro caso de sobreajuste. Al carecer de visión geométrica por el aplanado de los datos, y al tener tanta capacidad de memoria, la red optó por memorizar los tableros exactos del entrenamiento, perdiendo toda capacidad de generalizar reglas estratégicas.

A pesar de esta limitación, el **Modelo B** demostró ser la aproximación más estable y fue seleccionado como el modelo definitivo (`otelo_B.keras`) para competir en el torneo final.

IV-B. Torneo Final: IA Clásica vs IA Híbrida

Para evaluar empíricamente el impacto de la red neuronal en la toma de decisiones, se diseñó un entorno de pruebas automatizado (`torneo.py`) donde se enfrentaron dos agentes:

- **Agente Clásico:** Implementación de Minimax con poda alfa-beta utilizando la heurística manual de conteo de fichas.
- **Agente Híbrido:** Implementación de Minimax con poda alfa-beta, pero reemplazando la heurística manual por la red neuronal entrenada (`otelo_B.keras`).

Ambos agentes se configuraron con una profundidad máxima de búsqueda de 3 niveles. Para garantizar la equidad del enfrentamiento y mitigar la ventaja del primer movimiento, el torneo constó de 100 partidas consecutivas, garantizando que cada agente jugara 50 partidas con las fichas negras y 50 con las blancas.

Los resultados finales del torneo fueron un total de **72** victorias para el Agente Clásico, **23** victorias para el Agente Híbrido y **5** empates. Este resultado es coherente con el análisis previo: el Agente Clásico utiliza una heurística simple pero efectiva a niveles de profundidad bajos, mientras que el Agente Híbrido sufre de la falta de capacidad de generalización de la red neuronal que se comentó en la sección anterior. La red, al carecer de visión espacial, tiende a predecir empates, llevándolo a tomar decisiones inferiores frente a un oponente que evalúa de forma más directa la diferencia de fichas.

Sin embargo, el hecho de que el Agente Híbrido haya logrado 23 victorias y 5 empates frente a un oponente clásico indica que la red neuronal ha aprendido ciertos patrones estratégicos. La red neuronal no selecciona movimientos al

azar, demostrando el éxito funcional de la integración de Deep Learning en el proceso de toma de decisiones.

V. CONCLUSIONES

VI. BIBLIOGRAFÍA

REFERENCIAS

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020, ch. 5, "Adversarial Search".
- [2] World Othello Federation, "Othello Rules," [Online]. Disponible en: <https://www.worldothello.org/about/about-othello/othello-rules>
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [4] Dpto. de Ciencias de la Computación e Inteligencia Artificial, "Redes neuronales", Universidad de Sevilla, Apuntes de la asignatura Inteligencia Artificial, Curso 2025/2026.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.